d_08 社内OSスタンドアロン強化設計

> **目的**: 外部SaaS（freee/SmartHR/kintone/Salesforce）への依存を排除し、
> Supabase + LLMのみで完結する「完全自立型社内OS」を実現する。
>
> **方針**: CloudSign（電子署名）/ Bank API（銀行明細）/ gBizINFO（法人番号）の3つのみ外部API。
> それ以外は全て自前DBとパイプラインで完結させる。
>
> **最終更新**: 2026-03-29

---

0. パイプライン層の3層分離原則

> d_07 Section 1 のアーキテクチャ図と連動。

0.1 Layer A: 全社共通基盤（Base OS）— 36本

テナント登録時に**自動有効化**。どの業界の企業でも必ず使う。

領域	本数	内容
営業SFA/CRM/CS	12本	全業種で営業活動は発生。業種アウェア（業種判定・テンプレ選択）
バックオフィス	24本	全業種で経理・人事・総務は発生。業界非依存

0.2 Layer B: 業界特化プラグイン（Industry Plugin）

テナント登録時に**業界を選択**して有効化。各プラグインは**自己完結**。

- 独自のデータモデル（estimation_items, mfg_quotes等）
- 独自のエンジン（EstimationEngine, QuotingEngine等）
- 独自のルール（建設業法、ISO 9001等 ← micro/compliance.pyから移動）
- 独自のテンプレート（見積書、安全書類等）
- **他業界プラグインとは完全隔離。Layer Aとも直接依存しない。**

0.3 Layer C: 共有マイクロエージェント（Shared Primitives）— 19個

業界中立の原子操作のみ。業界固有ロジックは持たない。

- `compliance.py` → 汎用 `check_rules(data, rules)` のみ。建設業法ルールは建設プラグインに移動。
- `pdf_generator.py` → 汎用PDF生成のみ。営業テンプレはLayer A営業に移動。
- `rule_matcher.py` → 汎用マッチング。業界ヒントは各プラグインに移動。

0.4 TaskRouterの2層レジストリ

```
BASE_PIPELINES = { ... }      # Layer A: 全テナント有効（36本）
INDUSTRY_PIPELINES = {         # Layer B: 業界選択で有効化
    "construction": { ... },   # 建設8本
    "manufacturing": { ... },  # 製造8本
    ...
}

# 実行時:
# 1. BASE_PIPELINES を先にチェック
# 2. なければ INDUSTRY_PIPELINES[tenant.industry] をチェック
# 3. テナントの業界と不一致なら実行拒否
```

0.5 ディレクトリ構造（目標）

```
shachotwo-app/workers/
├── base/                    # Layer A: 全社共通基盤
│   ├── sales/               #   営業SFA/CRM/CS (12本)
│   │   ├── marketing/
│   │   ├── sfa/
│   │   ├── crm/
│   │   ├── cs/
│   │   ├── learning/
│   │   └── templates/       #   営業固有テンプレ（←microから移動）
│   └── backoffice/          #   バックオフィス (24本)
│       ├── accounting/      #     経理8本
│       ├── labor/           #     労務5本
│       ├── hr/              #     人事3本
│       ├── admin/           #     総務3本
│       ├── procurement/     #     調達2本
│       ├── legal/           #     法務2本
│       └── it/              #     IT1本
│
├── industry/                # Layer B: 業界特化プラグイン
│   ├── construction/        #   建設（models/engine/rules/pipelines/templates）
│   ├── manufacturing/       #   製造（models/engine/plugins/rules/pipelines/templates）
│   ├── clinic/              #   医療
│   ├── nursing/             #   介護
│   ├── realestate/          #   不動産
│   └── logistics/           #   物流
│
├── micro/                   # Layer C: 業界中立マイクロ
│   ├── ocr.py / extractor.py / calculator.py / validator.py
│   ├── generator.py / message.py / diff.py / table_parser.py
│   ├── compliance.py        #   汎用check_rules()のみ
│   ├── rule_matcher.py      #   汎用マッチングのみ
│   └── ...
│
└── connector/               # SaaS連携（業界中立）
```

> **注意**: 現在のコードはまだ旧構造（`workers/bpo/` 配下にフラット）。
> コードのリファクタリングは別タスクで実施。本セクションは**設計方針**の定義。

---

1. 現状の外部SaaS依存と排除方針

外部SaaS	現在の依存内容	排除方針	残す理由
freee	仕訳/請求書/入金/給与	**全排除** → 自前GL+請求+給与	—
SmartHR	従業員マスタ/社保届出	**全排除** → 自前従業員DB+届出生成	—
kintone	案件管理/業務DB	**全排除** → Supabaseで完結	—
Salesforce	CRM	**既に不要** → 自前leads/customers	—
CloudSign	電子署名	**維持**	法的効力は認定事業者のみ
Bank API	銀行明細取得	**維持**（CSV手動もOK）	銀行データは銀行のみ保有
gBizINFO	法人番号検索	**維持**	政府データは政府のみ

2. 追加DBテーブル設計（6領域25テーブル）

2.1 総勘定元帳（GL）— freee会計を完全置換

```
-- 勘定科目マスタ
CREATE TABLE chart_of_accounts (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    company_id UUID NOT NULL REFERENCES companies(id),
    account_code VARCHAR(10) NOT NULL,       -- "1100", "4100", "5100"
    account_name TEXT NOT NULL,              -- "現金", "売上高", "給与手当"
    account_type VARCHAR(20) NOT NULL,       -- asset/liability/equity/revenue/expense
    parent_code VARCHAR(10),                 -- 階層構造（上位科目）
    is_system BOOLEAN DEFAULT FALSE,         -- システム予約科目（削除不可）
    tax_category VARCHAR(20),               -- 課税/非課税/不課税/免税
    is_active BOOLEAN DEFAULT TRUE,
    display_order INT DEFAULT 0,
    created_at TIMESTAMPTZ DEFAULT now(),
    UNIQUE(company_id, account_code)
);

-- 仕訳帳（全取引の原本）
CREATE TABLE journal_entries (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    company_id UUID NOT NULL REFERENCES companies(id),
    entry_number SERIAL,                     -- 仕訳番号（自動採番）
    entry_date DATE NOT NULL,                -- 取引日
    description TEXT,                        -- 摘要
    source_type VARCHAR(30),                 -- "expense" / "invoice" / "payroll" /
"manual"
    source_id UUID,                          -- 元パイプラインの実行ID
    status VARCHAR(20) DEFAULT 'posted',     -- draft/posted/reversed
    posted_by UUID REFERENCES users(id),
    posted_at TIMESTAMPTZ,
    created_at TIMESTAMPTZ DEFAULT now()
);

-- 仕訳明細（借方・貸方行）
CREATE TABLE journal_entry_lines (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    journal_entry_id UUID NOT NULL REFERENCES journal_entries(id) ON DELETE CASCADE,
    line_number INT NOT NULL,
    account_code VARCHAR(10) NOT NULL,       -- 勘定科目コード
    debit_amount DECIMAL(15,2) DEFAULT 0,    -- 借方金額
    credit_amount DECIMAL(15,2) DEFAULT 0,   -- 貸方金額
    tax_amount DECIMAL(15,2) DEFAULT 0,      -- 消費税額
    tax_rate DECIMAL(5,2),                   -- 税率（10%, 8%）
    department_code VARCHAR(10),             -- 部門コード（原価管理用）
    description TEXT,                        -- 行摘要
    CONSTRAINT chk_debit_or_credit CHECK (
```

```
        (debit_amount > 0 AND credit_amount = 0) OR
        (debit_amount = 0 AND credit_amount > 0)
    )
);

-- 月次残高テーブル（高速集計用）
CREATE TABLE account_balances (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    company_id UUID NOT NULL REFERENCES companies(id),
    account_code VARCHAR(10) NOT NULL,
    fiscal_year INT NOT NULL,
    fiscal_month INT NOT NULL,               -- 1-12
    opening_balance DECIMAL(15,2) DEFAULT 0, -- 期首残高
    debit_total DECIMAL(15,2) DEFAULT 0,     -- 当月借方合計
    credit_total DECIMAL(15,2) DEFAULT 0,    -- 当月貸方合計
    closing_balance DECIMAL(15,2) DEFAULT 0, -- 期末残高
    updated_at TIMESTAMPTZ DEFAULT now(),
    UNIQUE(company_id, account_code, fiscal_year, fiscal_month)
);
```

ビュー:

```
-- 試算表ビュー
CREATE VIEW trial_balance AS
SELECT company_id, fiscal_year, fiscal_month,
       account_code, ca.account_name, ca.account_type,
       opening_balance, debit_total, credit_total, closing_balance
FROM account_balances ab
JOIN chart_of_accounts ca USING (company_id, account_code);

-- 損益計算書ビュー（revenue - expense）
-- 貸借対照表ビュー（asset = liability + equity）
```

2.2 従業員マスタ — SmartHR完全置換

```
-- 部門マスタ
CREATE TABLE departments (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    company_id UUID NOT NULL REFERENCES companies(id),
    department_code VARCHAR(10) NOT NULL,
    department_name TEXT NOT NULL,
    parent_id UUID REFERENCES departments(id),
    manager_id UUID,                          -- 部門長（employees.id）
    is_active BOOLEAN DEFAULT TRUE,
    UNIQUE(company_id, department_code)
);

-- 役職マスタ
CREATE TABLE positions (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    company_id UUID NOT NULL REFERENCES companies(id),
    position_code VARCHAR(10) NOT NULL,
    position_name TEXT NOT NULL,              -- "部長", "課長", "主任"
    grade INT DEFAULT 1,                      -- 等級
    UNIQUE(company_id, position_code)
);

-- 従業員マスタ（全業界共通）
CREATE TABLE employees (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    company_id UUID NOT NULL REFERENCES companies(id),
    employee_number VARCHAR(20) NOT NULL,     -- 社員番号
    last_name TEXT NOT NULL,
    first_name TEXT NOT NULL,
    last_name_kana TEXT,
    first_name_kana TEXT,
    birth_date DATE,
    gender VARCHAR(10),
    email TEXT,
    phone TEXT,
    address JSONB,                            -- {postal_code, prefecture, city, street}
    -- 雇用情報
    hire_date DATE NOT NULL,
    employment_type VARCHAR(20) NOT NULL,     -- full_time/part_time/contract/dispatch
    department_id UUID REFERENCES departments(id),
    position_id UUID REFERENCES positions(id),
    -- 給与情報
    base_salary DECIMAL(12,2),               -- 基本給（月額 or 時給）
    salary_type VARCHAR(10) DEFAULT 'monthly', -- monthly/hourly/daily
    commute_allowance DECIMAL(10,2) DEFAULT 0,
    -- 社保情報
    health_insurance_number VARCHAR(20),
    pension_number VARCHAR(20),
    employment_insurance_number VARCHAR(20),
```

```
    standard_monthly_remuneration DECIMAL(12,2), -- 標準報酬月額
    -- マイナンバー（暗号化必須）
    my_number_encrypted TEXT,                 -- AES-256-GCM暗号化
    -- 状態
    status VARCHAR(20) DEFAULT 'active',      -- active/on_leave/retired
    retirement_date DATE,
    retirement_reason VARCHAR(50),
    -- 有給
    pto_balance DECIMAL(5,1) DEFAULT 0,       -- 有給残日数
    pto_granted_date DATE,                    -- 付与日
    -- メタ
    created_at TIMESTAMPTZ DEFAULT now(),
    updated_at TIMESTAMPTZ DEFAULT now(),
    UNIQUE(company_id, employee_number)
);

-- 扶養家族
CREATE TABLE dependents (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    employee_id UUID NOT NULL REFERENCES employees(id) ON DELETE CASCADE,
    relationship VARCHAR(20) NOT NULL,        -- spouse/child/parent
    name TEXT NOT NULL,
    birth_date DATE,
    annual_income DECIMAL(12,2) DEFAULT 0,   -- 年収（扶養判定用）
    is_tax_dependent BOOLEAN DEFAULT TRUE,    -- 税扶養
    is_insurance_dependent BOOLEAN DEFAULT TRUE -- 社保扶養
);
```

2.3 給与・社保記録 — freee人事労務を完全置換

```
-- 給与計算結果
CREATE TABLE payroll_runs (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    company_id UUID NOT NULL REFERENCES companies(id),
    pay_period_year INT NOT NULL,
    pay_period_month INT NOT NULL,
    pay_date DATE NOT NULL,                  -- 支給日
    status VARCHAR(20) DEFAULT 'draft',      -- draft/calculated/approved/paid
    total_gross DECIMAL(15,2) DEFAULT 0,
    total_deductions DECIMAL(15,2) DEFAULT 0,
    total_net DECIMAL(15,2) DEFAULT 0,
    employee_count INT DEFAULT 0,
    approved_by UUID REFERENCES users(id),
    approved_at TIMESTAMPTZ,
    journal_entry_id UUID REFERENCES journal_entries(id), -- GL連動
    created_at TIMESTAMPTZ DEFAULT now(),
    UNIQUE(company_id, pay_period_year, pay_period_month)
);

-- 給与明細（従業員別）
CREATE TABLE payroll_items (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    payroll_run_id UUID NOT NULL REFERENCES payroll_runs(id) ON DELETE CASCADE,
    employee_id UUID NOT NULL REFERENCES employees(id),
    -- 支給
    base_salary DECIMAL(12,2) DEFAULT 0,
    overtime_pay DECIMAL(12,2) DEFAULT 0,
    overtime_hours DECIMAL(6,2) DEFAULT 0,
    late_night_pay DECIMAL(12,2) DEFAULT 0,
    holiday_pay DECIMAL(12,2) DEFAULT 0,
    commute_allowance DECIMAL(10,2) DEFAULT 0,
    other_allowances DECIMAL(12,2) DEFAULT 0,
    gross_pay DECIMAL(12,2) DEFAULT 0,
    -- 控除
    health_insurance DECIMAL(10,2) DEFAULT 0,
    pension DECIMAL(10,2) DEFAULT 0,
    employment_insurance DECIMAL(10,2) DEFAULT 0,
    income_tax DECIMAL(10,2) DEFAULT 0,
    resident_tax DECIMAL(10,2) DEFAULT 0,
    other_deductions DECIMAL(10,2) DEFAULT 0,
    total_deductions DECIMAL(12,2) DEFAULT 0,
    -- 差引支給額
    net_pay DECIMAL(12,2) DEFAULT 0,
    -- 勤怠サマリ
    work_days INT DEFAULT 0,
    absence_days INT DEFAULT 0,
    paid_leave_used DECIMAL(4,1) DEFAULT 0,
    -- 36協定
    monthly_overtime_hours DECIMAL(6,2) DEFAULT 0,
```

```
    yearly_overtime_hours DECIMAL(8,2) DEFAULT 0,
    overtime_alert BOOLEAN DEFAULT FALSE
);

-- 社保届出記録
CREATE TABLE social_insurance_filings (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    company_id UUID NOT NULL REFERENCES companies(id),
    employee_id UUID NOT NULL REFERENCES employees(id),
    filing_type VARCHAR(30) NOT NULL,         --
acquisition/loss/standard_base/monthly_change/bonus/maternity
    filing_date DATE,
    effective_date DATE,
    standard_monthly_pay DECIMAL(12,2),
    health_insurance_grade INT,
    pension_grade INT,
    status VARCHAR(20) DEFAULT 'draft',       -- draft/submitted/accepted/rejected
    document_pdf_path TEXT,
    submitted_at TIMESTAMPTZ,
    created_at TIMESTAMPTZ DEFAULT now()
);
```

2.4 勤怠記録 — KING OF TIME/ジョブカンを完全置換

```
-- 日次勤怠記録
CREATE TABLE timecards (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    company_id UUID NOT NULL REFERENCES companies(id),
    employee_id UUID NOT NULL REFERENCES employees(id),
    work_date DATE NOT NULL,
    clock_in TIMESTAMPTZ,                    -- 出勤打刻
    clock_out TIMESTAMPTZ,                   -- 退勤打刻
    break_minutes INT DEFAULT 60,            -- 休憩時間（分）
    work_minutes INT GENERATED ALWAYS AS (
        CASE WHEN clock_in IS NOT NULL AND clock_out IS NOT NULL
        THEN EXTRACT(EPOCH FROM (clock_out - clock_in))/60 - break_minutes
        ELSE 0 END
    ) STORED,
    overtime_minutes INT DEFAULT 0,          -- 残業時間（分）
    late_night_minutes INT DEFAULT 0,        -- 深夜時間（22-05時、分）
    holiday_work BOOLEAN DEFAULT FALSE,      -- 休日出勤
    absence_type VARCHAR(20),                -- null/paid_leave/sick/absent/special
    note TEXT,
    approved BOOLEAN DEFAULT FALSE,
    UNIQUE(company_id, employee_id, work_date)
);

-- シフトテンプレート
CREATE TABLE shift_templates (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    company_id UUID NOT NULL REFERENCES companies(id),
    template_name TEXT NOT NULL,
    start_time TIME NOT NULL,                -- 始業時刻
    end_time TIME NOT NULL,                  -- 終業時刻
    break_minutes INT DEFAULT 60,
    is_default BOOLEAN DEFAULT FALSE
);
```

2.5 銀行・支払 — 振込ファイル自前生成

```
-- 銀行口座マスタ
CREATE TABLE bank_accounts (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    company_id UUID NOT NULL REFERENCES companies(id),
    bank_name TEXT NOT NULL,
    branch_name TEXT NOT NULL,
    account_type VARCHAR(10) NOT NULL,        -- ordinary/checking/savings
    account_number VARCHAR(20) NOT NULL,
    account_holder TEXT NOT NULL,
    is_primary BOOLEAN DEFAULT FALSE,
    balance DECIMAL(15,2) DEFAULT 0,          -- 最終確認残高
    balance_as_of TIMESTAMPTZ
);

-- 銀行明細（CSVインポート or Bank API）
CREATE TABLE bank_statements (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    company_id UUID NOT NULL REFERENCES companies(id),
    bank_account_id UUID NOT NULL REFERENCES bank_accounts(id),
    transaction_date DATE NOT NULL,
    value_date DATE,
    description TEXT,                         -- 摘要
    debit_amount DECIMAL(15,2) DEFAULT 0,     -- 出金
    credit_amount DECIMAL(15,2) DEFAULT 0,    -- 入金
    balance DECIMAL(15,2),                    -- 残高
    is_reconciled BOOLEAN DEFAULT FALSE,      -- 消込済み
    matched_journal_id UUID REFERENCES journal_entries(id),
    import_batch_id UUID,                     -- CSVインポートバッチID
    created_at TIMESTAMPTZ DEFAULT now()
);

-- 振込指示（全銀フォーマット出力用）
CREATE TABLE payment_transfers (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    company_id UUID NOT NULL REFERENCES companies(id),
    transfer_type VARCHAR(20) NOT NULL,       -- payroll/vendor/tax/other
    source_id UUID,                           -- payroll_runs.id or ap_invoices.id
    from_bank_account_id UUID REFERENCES bank_accounts(id),
    to_bank_name TEXT NOT NULL,
    to_branch_name TEXT NOT NULL,
    to_account_type VARCHAR(10) NOT NULL,
    to_account_number VARCHAR(20) NOT NULL,
    to_account_holder TEXT NOT NULL,
    amount DECIMAL(15,2) NOT NULL,
    transfer_date DATE NOT NULL,
    status VARCHAR(20) DEFAULT 'pending',     -- pending/approved/sent/completed/failed
    zengin_file_path TEXT,                    -- 全銀フォーマットファイルパス
    approved_by UUID REFERENCES users(id),
```

```
    created_at TIMESTAMPTZ DEFAULT now()
);
```

2.6 在庫管理 — kintone/ERPを完全置換

```
-- 在庫品目マスタ
CREATE TABLE inventory_items (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    company_id UUID NOT NULL REFERENCES companies(id),
    item_code VARCHAR(30) NOT NULL,
    item_name TEXT NOT NULL,
    category VARCHAR(50),                    -- 原材料/仕掛品/製品/消耗品
    unit VARCHAR(10) DEFAULT '個',           -- 個/kg/m/本/箱
    standard_cost DECIMAL(12,2),             -- 標準原価
    reorder_point DECIMAL(12,2) DEFAULT 0,   -- 発注点
    safety_stock DECIMAL(12,2) DEFAULT 0,    -- 安全在庫
    is_active BOOLEAN DEFAULT TRUE,
    UNIQUE(company_id, item_code)
);

-- 在庫残高
CREATE TABLE stock_levels (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    company_id UUID NOT NULL REFERENCES companies(id),
    item_id UUID NOT NULL REFERENCES inventory_items(id),
    warehouse VARCHAR(50) DEFAULT 'main',    -- 倉庫名
    quantity DECIMAL(12,3) NOT NULL DEFAULT 0,
    last_counted_at TIMESTAMPTZ,
    updated_at TIMESTAMPTZ DEFAULT now(),
    UNIQUE(company_id, item_id, warehouse)
);

-- 在庫移動記録
CREATE TABLE stock_movements (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    company_id UUID NOT NULL REFERENCES companies(id),
    item_id UUID NOT NULL REFERENCES inventory_items(id),
    movement_type VARCHAR(20) NOT NULL,      -- receipt/issue/adjustment/transfer
    quantity DECIMAL(12,3) NOT NULL,          -- +入庫/-出庫
    warehouse VARCHAR(50) DEFAULT 'main',
    reference_type VARCHAR(30),              -- purchase_order/production_order/manual
    reference_id UUID,
    note TEXT,
    created_at TIMESTAMPTZ DEFAULT now()
);
```

---

3. 全銀フォーマット生成（新規マイクロエージェント）

```
新規ファイル: workers/micro/zengin_generator.py

機能: payment_transfers テーブルのレコードから全銀協フォーマット(固定長120バイト)の
      振込ファイルを生成する。

レコードフォーマット:
  ヘッダ(1): 種別=1, 振込依頼人コード, 振込依頼人名, 振込日, 仕向銀行
  データ(N): 種別=2, 被仕向銀行, 支店, 口座種別, 口座番号, 受取人名, 振込金額
  トレーラ(1): 種別=8, 合計件数, 合計金額
  エンド(1): 種別=9

出力: UTF-8 or Shift_JIS テキストファイル（銀行に応じて切替）
```

---

4. パイプライン修正マップ

4.1 freee依存の排除

パイプライン	現在	変更後
invoice_issue	saas_writer → freee API	Supabase `bpo_invoices` + `journal_entries` に直接書込
ar_management	saas_reader → freee 未入金	Supabase `bpo_invoices` WHERE status='sent' + `bank_statements`
ap_management	saas_reader → freee 買掛	Supabase `bpo_invoices` (受領) + `payment_transfers` 生成
bank_reconciliation	saas_reader → freee 帳簿	Supabase `journal_entries` + `bank_statements`
journal_entry	saas_writer → freee 仕訳API	Supabase `journal_entries` + `journal_entry_lines` に直接書込
monthly_close	saas_reader → freee 試算表	Supabase `account_balances` ビューから集計
tax_filing	saas_reader → freee 年次	Supabase `journal_entries` + `account_balances`

4.2 SmartHR依存の排除

パイプライン	現在	変更後
payroll	saas_reader → SmartHR従業員	Supabase `employees` + `timecards`
social_insurance	saas_reader → SmartHR + freee	Supabase `employees` + `payroll_items`
employee_onboarding	saas_writer → SmartHR登録	Supabase `employees` INSERT + SaaSアカウント作成
employee_offboarding	saas_writer → SmartHR更新	Supabase `employees` UPDATE status='retired'
year_end_adjustment	saas_reader → freee給与	Supabase `payroll_items` 年間集計
attendance	saas_reader → SmartHR勤怠	Supabase `timecards`
labor_compliance	saas_reader → 勤怠+給与	Supabase `timecards` + `payroll_items`

---

5. 勘定科目テンプレート（ゲノム方式）

業界ごとの勘定科目テンプレートをゲノムJSONで提供:

```
brain/genome/data/gl/
  ├── construction.json     # 建設業会計（完成工事高/未成工事支出金/工事原価）
  ├── manufacturing.json    # 製造業会計（製造原価/仕掛品/原材料）
  ├── service.json          # サービス業（売上/外注費/人件費）
  └── common.json           # 共通科目（現金/預金/売掛/買掛/租税公課）
```

テナント登録時に `genome/applicator` が勘定科目を `chart_of_accounts` に一括登録。

---

6. 自前OS化後のアーキテクチャ

```
┌──────────────────────────────────────────────────────────────┐
│                    シャチョツー社内OS                          │
│                     （完全自立型）                             │
│                                                                │
│  ┌──────────────────────────────────────────────────────┐    │
│  │ 自前で完結（Supabase + LLM）                          │    │
│  │                                                       │    │
│  │  会計GL      → chart_of_accounts + journal_entries   │    │
│  │  給与計算    → employees + timecards + payroll_*     │    │
│  │  社保届出    → social_insurance_filings              │    │
│  │  CRM/SFA     → leads + opportunities + customers     │    │
│  │  CS/サポート → tickets + health_scores               │    │
│  │  在庫管理    → inventory_items + stock_movements     │    │
│  │  銀行振込    → payment_transfers + zengin生成        │    │
│  │  経費精算    → bpo_expenses + journal_entries        │    │
│  │  勤怠管理    → timecards + shift_templates           │    │
│  │  業界BPO     → estimation/quoting/quality/etc        │    │
│  │  ナレッジ    → knowledge_items + embeddings          │    │
│  └──────────────────────────────────────────────────────┘    │
│                                                                │
│  ┌──────────────────────────────────────────────────────┐    │
│  │ 外部API（3件のみ、代替不可）                          │    │
│  │                                                       │    │
│  │  CloudSign → 法的電子署名（契約締結時のみ）           │    │
│  │  Bank API  → 銀行明細取得（CSV手動もOK）              │    │
│  │  gBizINFO  → 法人番号検索（反社チェック時のみ）       │    │
│  └──────────────────────────────────────────────────────┘    │
│                                                                │
│  ┌──────────────────────────────────────────────────────┐    │
│  │ オプション同期（接続すればデータ同期、なくても動く）  │    │
│  │                                                       │    │
│  │  freee       → GL双方向同期（税理士が使う場合）       │    │
│  │  SmartHR     → 従業員データ同期（既存利用の場合）     │    │
│  │  Slack       → 通知送信（なければログに記録）         │    │
│  │  kintone     → 既存業務DB同期（移行期間のみ）         │    │
│  └──────────────────────────────────────────────────────┘    │
└──────────────────────────────────────────────────────────────┘
```

7. 実装順序

Phase 2a（即実行 — DB基盤）

1. マイグレーション024: GL4テーブル（chart_of_accounts, journal_entries, journal_entry_lines, account_balances）
2. マイグレーション025: 従業員5テーブル（departments, positions, employees, dependents）
3. マイグレーション026: 給与3テーブル（payroll_runs, payroll_items, social_insurance_filings）
4. マイグレーション027: 勤怠2テーブル（timecards, shift_templates）
5. マイグレーション028: 銀行3テーブル（bank_accounts, bank_statements, payment_transfers）
6. マイグレーション029: 在庫3テーブル（inventory_items, stock_levels, stock_movements）

Phase 2b（パイプライン修正）

7. 勘定科目ゲノムJSON（4業界分）
8. journal_entry_pipeline修正（freee→Supabase直接書込）
9. payroll_pipeline修正（SmartHR→Supabase employees）
10. attendance_pipeline修正（CSV→Supabase timecards）
11. ar/ap/bank_reconciliation修正（freee→Supabase）
12. zengin_generator マイクロエージェント新規作成

Phase 2c（新規ルーター）

13. /api/v1/accounting/ — GL操作（仕訳登録/試算表/BS/PL）
14. /api/v1/hr/ — 従業員CRUD/部門/役職
15. /api/v1/attendance/ — 打刻/勤怠一覧
16. /api/v1/inventory/ — 在庫操作

---

8. 外部SaaSは「オプション同期」として残す

既存のコネクタは削除しない。接続されていれば同期する「ブリッジモード」を提供:

```
# 例: journal_entry_pipeline 修正後
async def run_journal_entry_pipeline(company_id, input_data, **kwargs):
    # Step 1: Supabaseに仕訳を書き込む（必須）
    await write_to_supabase_gl(company_id, entries)

    # Step 2: freee接続がある場合のみ同期（オプション）
    if await has_active_connector(company_id, "freee"):
        await sync_to_freee(company_id, entries)
```

これにより:
- freeeを使っていない企業 → Supabaseのみで完結
- freeeを使っている企業 → 自動同期（税理士がfreeeで確認できる）
- 移行期間 → 両方に書き込み、freeeの値と照合

---

8. 法定帳簿・法定調書・コンプライアンス自動化

> 詳細設計: `b_詳細設計/b_09_法定帳簿・法定調書・コンプライアンス設計.md`

8.1 対象法定帳簿（4種、法定保存義務あり）

帳簿	根拠法令	保存義務	罰則	新規テーブル
賃金台帳	労基法108条	5年（経過措置3年）	¥30万	`wage_ledger`
労働者名簿	労基法107条	5年（退職日起算）	¥30万	`worker_registry`
出勤簿	労基法109条解釈通達	5年	是正勧告	`timecards` 拡張
有給休暇管理簿	労基法施行規則24条の7	5年	¥30万/人	`paid_leave_grants` + `paid_leave_usage`

8.2 対象法定調書（6種、提出義務あり）

調書	提出先	提出期限	罰則	新規テーブル
源泉徴収票	税務署 + 本人	翌年1/31	¥50万	`withholding_slips`
給与支払報告書	市区町村	翌年1/31	¥10万過料	`salary_payment_reports`
法定調書合計表	税務署	翌年1/31	¥50万	`legal_report_summaries`
労働保険年度更新	労働局	7/10	追徴金10%	`labor_insurance_reports`
36協定届	労基署	有効期間開始前	6ヶ月懲役/¥30万	`overtime_agreements`
社保各届出	年金事務所	事実から5日以内	6ヶ月懲役/¥50万	`social_insurance_notifications`

8.3 年間コンプライアンスカレンダー

月	義務	期限	パイプライン
毎月	源泉所得税納付	翌月10日	payroll
毎月	社会保険料納付	翌月末	social_insurance
毎月	住民税特別徴収納付	翌月10日	payroll
1月	源泉徴収票/給与支払報告書/法定調書合計表	1/31	withholding_slip / salary_report / legal_report_summary
4月	社会保険料率改定反映	4月支給分〜	social_insurance
6-7月	労働保険年度更新	7/10	labor_insurance
7月	算定基礎届	7/10	social_insurance
10月	最低賃金改定チェック	10月発効	labor_compliance
11-12月	年末調整	12月最終給与	year_end_adjustment
随時	入退社届出	事実から5日以内	social_insurance

8.4 新規DBテーブル（法定帳簿・調書用: 13テーブル）

`wage_ledger` / `worker_registry` / `paid_leave_grants` / `paid_leave_usage` / `social_insurance_notifications` / `standard_remuneration` / `overtime_agreements` / `overtime_monitoring` / `withholding_slips` / `salary_payment_reports` / `legal_report_summaries` / `labor_insurance_reports` / `labor_insurance_rates`

8.5 新規パイプライン（法定帳簿・調書用: 6本）

パイプライン	トリガー	出力
`paid_leave_pipeline`	日次+イベント	有給付与/使用記録、5日取得義務監視
`overtime_agreement_pipeline`	年次+日次監視	36協定ドラフト、時間外上限アラート
`withholding_slip_pipeline`	年次(12月)+退職時	源泉徴収票PDF
`salary_report_pipeline`	年次(1月)	給与支払報告書（市区町村別）
`legal_report_summary_pipeline`	年次(1月)	法定調書合計表
`labor_insurance_pipeline`	年次(6月)	労働保険概算・確定申告書

---

9. 電子帳簿保存法対応

> 詳細設計: `b_詳細設計/b_09_法定帳簿・法定調書・コンプライアンス設計.md`

9.1 方針

- Phase 1: 訂正削除履歴方式（コスト0、認定タイムスタンプ不要）
- 物理削除の完全禁止（DBトリガーでブロック）
- 検索3要件（取引年月日/取引先/金額）のインデックス付きカラム
- 7年保存の自動管理

9.2 新規テーブル

- `file_attachments` — 26カラム（ファイルメタデータ+検索要件+保存期間管理+バージョン管理）
- `file_attachment_audit_log` — 訂正削除履歴（INSERT ONLY）

---

10. 下請法対応

10.1 方針

- 資本金区分による適用判定の自動化
- 60日以内支払の3段階アラート（45日/55日/60日）
- 3条書面（発注書）の12必須項目チェック
- 遅延利息（年14.6%）の自動算出
- ap_management_pipelineへの組み込み

10.2 新規テーブル

- `subcontractor_profiles` — 下請事業者プロファイル（資本金/適用判定）
- `subcontract_transactions` — 下請取引管理（60日ルール監視付き）

---

11. インボイス制度対応

11.1 方針

- 国税庁適格請求書発行事業者公表API連携
- 登録番号の定期検証（月次バッチ）
- 経過措置の自動適用（2026/9まで80%→2029/9まで50%→以降0%）
- 適格請求書の6記載要件チェック
- 端数処理ルール（1請求書あたり税率ごとに1回）

11.2 新規テーブル

- `invoice_registration` — 取引先のインボイス登録番号管理
- `invoice_verification_log` — 国税庁API検証履歴

---

12. 承認ルールエンジン

> 詳細設計: `b_詳細設計/b_10_承認エンジン・エラーハンドリング・データ移行設計.md`

12.1 方針

- 宣言的ルール定義（JSONベース条件式）でコード修正不要
- 金額別多段階承認（¥1万→課長/¥5万→部長/¥50万→社長）
- 代理承認（delegation）+ タイムアウトエスカレーション（7日→14日→CEO）
- 職務分離の強制（給与計算者≠振込承認者）

12.2 新規テーブル

- `approval_rules` — 条件→承認フローマッピング
- `approval_steps` — 多段階承認の各ステップ
- `approval_delegations` — 代理承認の委任期間
- `approval_instances` — 実行中の承認インスタンス
- `approval_instance_steps` — インスタンスの各ステップ状態

---

13. パイプラインエラーハンドリング

13.1 方針

- DB単独操作: Supabase RPC（PostgreSQL関数）でトランザクション管理
- 外部API+DB: Sagaパターン（補償トランザクション）
- GL補償仕訳: 失敗時に自動で反対仕訳を生成
- 冪等性キー: `pipeline_type + company_id + input_hash` のSHA-256
- Dead Letter Queue: 3回失敗→手動対応キュー→Slack URGENT通知

13.2 新規テーブル

- `pipeline_idempotency` — 冪等性管理
- `pipeline_failures` — Dead Letter Queue
- `pipeline_step_logs` — ステップレベルの完了状態追跡

---

14. CSVインポート/データ移行

14.1 方針

- 6テーブル対応
  （employees/vendors/chart_of_accounts/inventory_items/bank_statements/timecards）

- エンコーディング自動検出（UTF-8/Shift_JIS/EUC-JP）
- 3段階バリデーション（構造→型→ビジネスルール）
- バッチ単位のロールバック可能
- freee/SmartHRからのAPI移行パス（科目マッピング+残高照合）

14.2 新規テーブル

- `import_batches` — インポート履歴・ロールバック管理

---

15. セキュリティ: マイナンバー・RLS・職務分離

> 詳細設計: `b_詳細設計/b_08_バックオフィス基盤設計_マイナンバー_RLS_UI.md`

15.1 マイナンバーライフサイクル

- 収集: 2段階本人確認、入力後即暗号化、画面に平文表示しない
- 保管: AES-256-GCM（専用鍵、一般PII鍵と分離）、`my_number_admin` ロールのみ復号可
- 利用: 3目的に厳格限定（源泉徴収票/社保届出/支払調書）
- 廃棄: 法定保存期間（最長7年）経過後、暗号化カラムをNULL更新、廃棄ログ永久保存

15.2 RLS 6ロールモデル

ロール	説明
admin	全権（ただしマイナンバー復号不可）
hr_manager	従業員マスタ/勤怠/有給のCRUD
finance	仕訳/請求/支払/銀行のCRUD
my_number_admin	マイナンバー復号権限（最小人数）
employee	自分のデータのみ閲覧
auditor	全データ読取専用（監査用）

15.3 職務分離（5つの必須SoD）

アクション対	制約
給与計算 ≠ 給与振込承認	approval_rulesで強制
仕訳入力 ≠ 仕訳承認	approval_rulesで強制
請求書作成 ≠ 入金消込	approval_rulesで強制
発注 ≠ 検収	approval_rulesで強制
従業員登録 ≠ 給与設定	approval_rulesで強制

15.4 PII暗号化拡大

マイナンバー以外にも暗号化: employees.phone / employees.address / bank_accounts.account_number / dependents.name+birth_date

---

16. フロントエンド7モジュール（50ページ）

> 詳細設計: `b_詳細設計/b_08_バックオフィス基盤設計_マイナンバー_RLS_UI.md`

モジュール	ページ数	主要機能	実装優先度
勤怠 `/attendance`	6p	打刻/タイムカード/月次集計/シフト/有給申請/年間管理	P0
給与 `/payroll`	7p	給与計算/明細/賞与/社保料/年末調整/源泉徴収/振込	P0
人事 `/hr`	8p	従業員一覧/マスタ/部門/役職/入社/退社/扶養/資格	P1
会計 `/accounting`	8p	勘定科目/仕訳入力/元帳/試算表/BS/PL/照合/決算	P1
銀行 `/banking`	6p	口座管理/明細インポート/照合/振込/全銀ファイル/支払	P2
在庫 `/inventory`	6p	品目マスタ/在庫照会/入庫/出庫/棚卸/発注点	P2
経営 `/executive`	9p	統合KPI/P&L/CF/HR/営業/BPO/コンプラ/予算/戦略	P3

---

17. 追加DBテーブル総括

d_08で定義した全テーブル（セクション2+8-14の合計）

セクション	テーブル数	主要テーブル
§2 GL	4	chart_of_accounts, journal_entries, journal_entry_lines, account_balances
§2 従業員	4	departments, positions, employees, dependents
§2 給与	3	payroll_runs, payroll_items, social_insurance_filings
§2 勤怠	2	timecards, shift_templates
§2 銀行	3	bank_accounts, bank_statements, payment_transfers
§2 在庫	3	inventory_items, stock_levels, stock_movements
§8 法定帳簿	13	wage_ledger, worker_registry, paid_leave_, *overtime_*, withholding_*, etc.
§9 電帳法	2	file_attachments, file_attachment_audit_log
§10 下請法	2	subcontractor_profiles, subcontract_transactions
§11 インボイス	2	invoice_registration, invoice_verification_log
§12 承認エンジン	5	approval_rules, approval_steps, approval_delegations, approval_instances, approval_instance_steps
§13 エラー処理	3	pipeline_idempotency, pipeline_failures, pipeline_step_logs
§14 インポート	1	import_batches
合計	**47**	

追加パイプライン（セクション8で定義: 6本）

paid_leave / overtime_agreement / withholding_slip / salary_report / legal_report_summary / labor_insurance